



Bachelor Informatik/Wirtschaftsinformatik

Integrationsprojekt Kryptographie Teil 2

Spezifikationsdokument

Hannover, 8. Juni 2026

Sommerquartal 2026

Nina Neumann, Johanna Amini, Amirreza Ahmadi Darani, Xuan-Loc Tran, Franz Müller, Tammo Lütten,
Jannes Breuer, Bjarne Möller, Darko Marjanovic, Oliver Knöbl, Rafael Ulmer

Anmerkung

Die vorliegende Arbeit wurde im Rahmen eines wissenschaftlichen Projekts an der Fachhochschule für die Wirtschaft (FHDW) Hannover erstellt. Alle in dieser Arbeit verwendeten Informationen, Daten und Ergebnisse sind ausschließlich für akademische Zwecke bestimmt. Die Autoren übernehmen keine Verantwortung für die Richtigkeit, Vollständigkeit oder Aktualität der in dieser Arbeit enthaltenen Informationen. Jegliche Nutzung der Inhalte dieser Arbeit erfolgt auf eigenes Risiko.

Erklärung der Selbstständigkeit

Hiermit versichern wir, dass wir das vorliegende Spezifikationsdokument selbständig verfasst und keine anderen als die angegebenen Quellen verwendet haben. Alle Textstellen und andere Inhalte wie Bilder, Quellcode oder Daten, die wörtlich oder sinngemäß aus anderen Quellen entnommen sind, haben wir eindeutig mit korrekten Quellenangaben versehen. Über Zitierrichtlinien sind wir schriftlich informiert worden. Diese Arbeit wurde bisher in gleicher oder ähnlicher Form, auch nicht in Teilen, keiner anderen Prüfungsbehörde vorgelegt.

Falls wir während der Erstellung dieser Arbeit generative KI oder andere KI-gestützte Technologien verwendet haben, die über grundlegende Werkzeuge für Übersetzungen und zur Überprüfung von Grammatik oder Rechtschreibung hinausgehen, erklären wir, nach der Nutzung dieser Tools den Inhalt unseres Spezifikationsdokuments überprüft und bearbeitet zu haben und übernehmen die volle Verantwortung für die diesbezüglichen Ausführungen. Eine Verwendung solcher Werkzeuge haben wir in unserer Arbeit dokumentiert (bspw. im Abschnitt „Struktur der Arbeit, Methodik“ oder durch Erläuterungen zur Nutzung im Anhang).

Inhaltsverzeichnis

1	Einleitung	3
2	Querschnittssektionen	4
2.1	Architektur-Übersich	4
2.2	Gateway und Routing	4
2.3	Key Lifecycle	4
2.4	Serialisierung	4
2.5	Build, Deployment, Tests	4
3	Client	5
4	Use-Cases	6
4.1	Encrypted-Key-Value-Store	6
4.2	Encrypted-Age-Verification	7
4.3	Encrypted-Voting-&-Polling	14
4.4	Sealed-Bid-Auction	15
4.5	Encrypted-Statistics-Service	16
4.6	Encrypted-Genomics	17
4.7	Encrypted-Image-Processing	18
4.8	Encrypted-Leaderboard	19
4.9	Encrypted-Program-Execution	20
A	Anhang	21

1 Einleitung

[Mus23]

2 Querschnittssektionen

2.1 Architektur-Übersich

2.2 Gateway und Routing

2.3 Key Lifecycle

2.4 Serialisierung

2.5 Build, Deployment, Tests

3 Client

4 Use-Cases

4.1 Encrypted-Key-Value-Store

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.2 Encrypted-Age-Verification

Funktionsbeschreibung

Der Use Case Encrypted Age Verification löst das Problem, das Alter einer Person serverseitig zu prüfen, ohne dass der Server den tatsächlichen Alterswert jemals im Klartext zu Gesicht bekommt. Das Ergebnis der Prüfung (volljährig: ja/nein) wird ebenfalls verschlüsselt zurückgegeben - der Server kennt weder Eingabe noch Ausgabe.

Das Alter wird bereits auf dem Client mit dem ClientKey verschlüsselt und ausschließlich in verschlüsselter Form an das Backend übertragen. Das Backend führt die Altersüberprüfung mittels Fully Homomorphic Encryption (TFHE) durch, ohne den zugrunde liegenden Alterswert entschlüsseln zu können. Die Entschlüsselung des Ergebnisses ist ausschließlich durch den Besitzer des ClientKeys möglich.

Akteure:

- Client: Generiert das TFHE-Schlüsselpaar, verschlüsselt das Alter mit dem ClientKey, sendet `encrypted_age` und `server_key` an den Server, empfängt das verschlüsselte Ergebnis und entschlüsselt es lokal. Der Client besitzt den ClientKey und ist der einzige Akteur, der das Ergebnis entschlüsseln kann.
- Backend (Server): Empfängt `encrypted_age` und `CompressedServerKey`, setzt den ServerKey, führt die homomorphe Altersüberprüfung (`age_check`) durch und gibt das verschlüsselte Ergebnis (`FheBool`) zurück. Der Server sieht zu keinem Zeitpunkt das Alter oder das Ergebnis im Klartext.

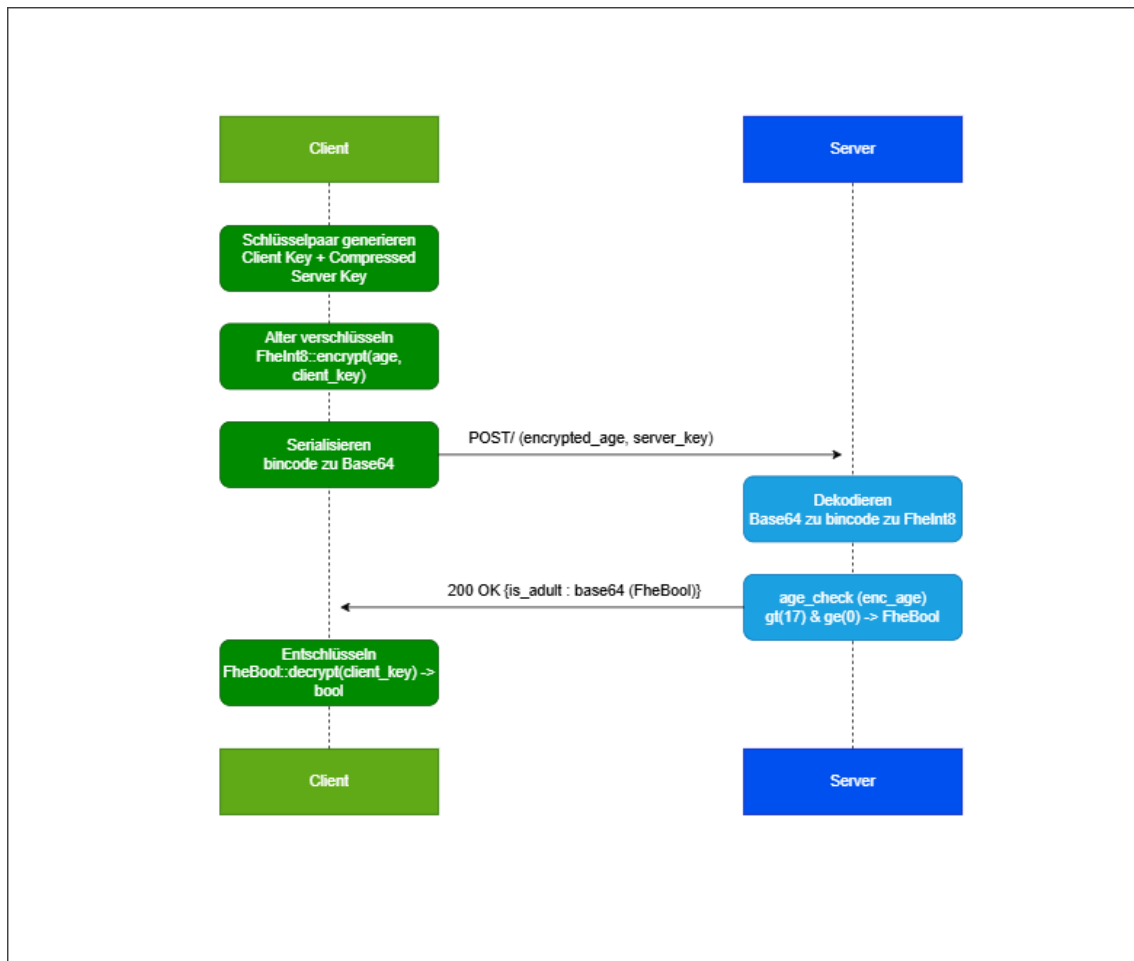
Lebenszyklus einer Session: Da dieser Use Case vollständig zustandslos ist, gibt es keine persistente Session. Der gesamte Ablauf findet innerhalb eines einzelnen HTTP-Requests statt:

1. Vorbereitung (Client): Client generiert `ClientKey` und `CompressedServerKey` lokal. Das Alter wird als `FheInt8` mit dem `ClientKey` verschlüsselt. Beide Werte werden bincode-serialisiert und base64-kodiert.
2. Request: Client sendet `POST` / mit `encrypted_age` und `server_key` als JSON-Body.
3. Verarbeitung (Server): Server dekodiert und deserialisiert beide Felder, setzt den `ServerKey` im globalen Thread-Kontext und führt `age_check` aus: `enc_age.gt(17) & enc_age.ge(0)`.
4. Response: Server serialisiert das `FheBool`-Ergebnis, base64-kodiert es und gibt es als `is_adult` zurück.
5. Auswertung (Client): Client dekodiert und deserialisiert `is_adult`, entschlüsselt das `FheBool` mit dem `ClientKey` und erhält das boolesche Ergebnis.

Verhaltensdiagramm:

OpenAPI-Schnittstelle

Der Service stellt eine einzelne verschlüsselte Verifikations-API bereit. Die OpenAPI-Definition wird automatisch generiert und ist unter `/openapi.json` sowie `/docs` (Swagger



UI) erreichbar.

Im gesamten Service wird der ServerKey als CompressedServerKey übertragen (bincode-serialisiert, base64-kodiert).

POST /: Führt eine verschlüsselte Altersverifikation durch.

Request:

```

{
  "encrypted_age": "string",
  "server_key": "string"
}

```

Response 200:

```

{
  "is_adult": "string"
}

```

is_adult ist ein base64-kodierter, bincode-serialisierter FheBool - true wenn Alter ≥ 18 und ≥ 0 .

Fehlercodes:

- 400 - Ungültiges Base64 oder beschädigtes bincode in server_key oder

encrypted_age

- 500 - Serialisierungsfehler beim Kodieren des Ergebnisses

Body-Limit: 2 GiB (notwendig wegen der Größe des CompressedServerKey)

Trust- und Threat-Model

	Server klar	Server verschlüsselt	Nur Client
Alter (numerischer Wert)		X	
Ergebnis (volljährig: ja/nein)		X	
ServerKey / CompressedServerKey	X		X
ClientKey			X

Analyse: Beobachtbare Metadaten: TFHE schützt ausschließlich den Inhalt des Alters und des Ergebnisses. Für den Server bleiben weiterhin verschiedene Metadaten sichtbar:

- Zeitpunkte und Häufigkeit der Anfragen
- IP-Adresse des anfragenden Clients
- Charakteristische Payload-Größe (80 MB), die auf TFHE-Schlüsselübertragung schließen lässt
- Anzahl der Anfragen pro IP

Da `age_check` eine datenunabhängige Berechnung ist (keine Branches auf verschlüsselten Werten), gibt es kein Timing-Side-Channel über die Rechendauer. Ein Server-Operator kann aus den Metadaten allenfalls Nutzungshäufigkeit ableiten, nicht jedoch das Alter einzelner Nutzer.

Restvertrauen in den Server: Der Server kann zwar das Alter nicht entschlüsseln, muss jedoch weiterhin als korrekter Ausführer der Verifikationslogik vertrauenswürdig sein. Insbesondere wird angenommen, dass der Server:

- `age_check` korrekt und unverändert ausführt (kein Austausch gegen eine Funktion, die immer `true` zurückgibt)
- Das korrekte `FheBool`-Ergebnis zurücksendet und es nicht durch einen manipulierten Ciphertext ersetzt
- Den ServerKey nicht persistiert oder weitergibt

TFHE reduziert somit die Vertrauensabhängigkeit hinsichtlich der Inhaltsvertraulichkeit, ersetzt jedoch kein vollständig vertrauensloses Protokoll.

Annahmen außerhalb von TFHE: Die Sicherheitsbetrachtung basiert auf folgenden Annahmen:

- Die Kommunikation zwischen Client und Server erfolgt über TLS.
- Der Client führt Verschlüsselung und Entschlüsselung korrekt aus.

- Die verwendete TFHE-rs-Bibliothek wird als kryptographisch korrekt implementierte Black Box betrachtet.
- Es gibt keine Authentifizierung am Endpunkt: Jeder, der einen gültigen `CompressedServerKey` besitzt, kann Anfragen stellen.

Schutzversprechen: Durch den Einsatz von TFHE wird garantiert:

- Der Server kann das Alter des Nutzers nicht im Klartext lesen.
- Der Server kann nicht feststellen, ob das Ergebnis `true` oder `false` ist.
- Das Alter wird ausschließlich verschlüsselt verarbeitet.

Nicht garantiert werden:

- Schutz vor einem Server, der `age_check` durch eine manipulierte Funktion ersetzt
- Schutz vor Traffic-Analyse (Payload-Größe, Timing)
- Authentizität des Clients (kein ZKP, dass der Client den `ServerKey` korrekt erzeugt hat)

Konkret bedeutet das: Der Server kennt nicht das Alter des Nutzers und kann nicht feststellen, ob das Ergebnis positiv oder negativ ausgefallen ist. Sichtbar bleiben ausschließlich Zeitpunkt, Herkunft und Häufigkeit der Anfragen.

FHE-Designentscheidungen

Verwendete TFHE-rs-Typen: Für das Alter wird `FheInt8` (vorzeichenbehaftetes 8-Bit-Integer, Wertebereich -128 bis 127) verwendet. Diese Wahl ergibt sich aus zwei Gründen:

Altersangaben in Jahren liegen typischerweise im Bereich 0-127, also weit innerhalb von `i8`. `FheUInt8` wäre für den positiven Ast ausreichend, aber `FheInt8` erlaubt es, negative Eingaben explizit als ungültig zu erkennen und abzufangen (s. `is_positive-Check`). Zudem unterstützt `FheInt8` `gt` und `ge` mit vorzeichenbehafteter Ganzzahlsemantik, was den Negativen-Grenzwert-Test (`ge(0)`) korrekt macht.

Das Ergebnis ist ein `FheBool` (verschlüsseltes Bit), was dem binären Charakter der Frage (volljährig: ja/nein) entspricht.

Verwendete homomorphe Operationen: Der Server führt ausschließlich zwei Vergleiche und eine boolesche Verknüpfung aus. Andere homomorphe Operationen werden nicht benötigt, dadurch bleibt die serverseitige Auswertung auf die minimal nötige Anzahl FHE-Operationen beschränkt.

Operation	Verwendung
<code>FheInt8::gt</code>	<code>enc_age > 17</code> → Alter ≥ 18
<code>FheInt8::ge</code>	<code>enc_age > 0</code> → kein negativer Wert
<code>FheBool::bitand</code>	Verknüpfung beider Bedingungen

Verworfen Alternativen: `FheUInt8` statt `FheInt8`

Wäre für rein positive Eingaben ausreichend. Verworfen, weil damit keine sinnvolle Behandlung negativer Eingaben möglich ist - `FheUInt8` interpretiert -1 als 255, was den Negativen-Grenzwert-Test (`age_check(-1)  $\rightarrow$  false`) unmöglich machen würde.

Schwellwert als verschlüsselter Parameter

Denkbar wäre, den Grenzwert (18) ebenfalls als `FheInt8` zu übergeben, um ihn serverseitig variabel zu halten. Verworfen, weil der Schwellwert in diesem Use Case keine schützenswerte Information ist und eine feste Konstante die Implementierung erheblich vereinfacht.

`FheInt32` oder größere Typen

Größere Bitbreiten würden höhere Latenzen pro homomorpher Operation verursachen, ohne Mehrwert - Altersangaben benötigen keine mehr als 8 Bit.

Komplexität der eigenen Algorithmen

Da der Use Case ausschließlich aus einer festen Anzahl homomorpher Operationen auf einem einzelnen Ciphertext besteht, gibt es keine parametrisierten Eingabegrößen. Die Komplexität aller Funktionen ist konstant.

Funktion	Zeitkomplexität	Platzkomplexität
<code>decode_server_key</code>	$O()$	$O(1)$
<code>decode_encrypted_age</code>	$O()$	$O(1)$
<code>age_check</code>	$O()$	$O(1)$
<code>encode_result</code>	$O()$	$O(1)$
<code>verify_age</code> (gesamt)	$O()$	$O(1)$

`verify_age` - $O(1)$, $O(1)$

Die Funktion führt genau zwei Vergleiche (`gt(17)`, `ge(0)`) und eine AND-Verknüpfung (`&`) auf einem `FheInt8`-Wert durch. Alle drei sind Operationen fester Bitbreite (8 Bit) auf einem einzelnen Ciphertext - unabhängig von jeder Eingabegröße. Gemäß der Konvention, homomorphe Operationen als $O(1)$ zu zählen, ist `age_check` $\in O(1)$.

Die De- und Serialisierungsschritte (`bincode`, `base64`) operieren auf Byte-Arrays fester Länge (durch die TFHE-rs-Typen bestimmt) und sind ebenfalls $O(1)$ bezüglich fachlicher Parameter.

Performance-Messung

Mess-Setup & Methodik

Die Performance- und Stresstests wurden auf einem virtuellen KVM-Server von Netcup durchgeführt. Die Last wurde extern mittels k6 von einer lokalen Windows-Maschine über das Internet injiziert.

Es wurde ein einziger relevanter Endpunkt getestet: `POST /`. Alle anderen Endpunkte (`/health`, `/docs`) stellen nur Grundrauschen dar und wurden nicht gemessen.

Die gemessene Gesamtlatenz setzt sich daher aus drei Anteilen zusammen:

1. Netzwerkübertragung des 80 MB ServerKey (dominanter Anteil bei Remote-Messung)
2. `CompressedServerKey::decompress()` - rechenintensive Dekomprimierung
3. `age_check()` - zwei FHE-Vergleiche + AND-Verknüpfung

Die gemessenen Latenzen spiegeln den realistischen End-to-End-Overhead des zustandslosen Designs wider.

- Tool: k6 v2.0.0
- TFHE: `ConfigBuilder::default()`
- Datum: 05.06.2026
- Server: Netcup KVM

Test 1 - Baseline (1 VU, 10 sequentielle Requests) (05.06.2026)

Metrik	Wert
p50	13.65 s
p90	29,26 s
p95	43,51 s
Maximum	60,00 s
Fehlerrate	0%
Durchsatz	0,05 req/s

Fazit von Test 1:

Bereits bei einem einzelnen sequentiellen Client ist die Latenz hoch. Der dominierende Faktor ist die Übertragung des 80 MB ServerKey über das Internet sowie dessen Dekomprimierung. Die hohe Varianz zwischen p50 (13,65 s) und p95 (43,51 s) deutet darauf hin, dass Netzwerkschwankungen einen erheblichen Einfluss haben.

Test 2 - Stresstest (ramping bis 10 VUs) (05.06.2026, 3 Durchläufe)

Der Test wurde dreimal unabhängig voneinander ausgeführt. Die Tabelle zeigt Mittelwerte sowie die beobachtete Spanne zur Einordnung der Varianz.

Metrik	Lauf 1	Lauf 2	Lauf 3	Mittelwert
p50	13,65 s	13,70 s	13,59 s	13,65 s
p90	37,23 s	39,76 s	27,02 s	34,67 s
p95	43,51 s	52,74 s	30,69 s	42,31 s
Maximum	60,00 s	54,56 s	42,78 s	52,45 s
Fehlerrate	30 %	26 %	35 %	30 %
Durchsatz	0,05 req/s	0,05 req/s	0,05 req/s	0,05 req/s

Fazit von Test 2:

Unter paralleler Last liegt die Fehlerrate im Mittel bei 30%. Die Fehler sind ausschließlich HTTP 499 (Client Closed Request) und HTTP 504 (Gateway Timeout) - der vorgelagerte Nginx-Proxy trennt Verbindungen nach 60 s, bevor der Server die Verarbeitung abschließen kann. Der Server selbst lief in allen drei Durchläufen vollständig stabil und verarbeitete alle Anfragen ohne eigene Fehler.

Der p50 ist mit 13,6 s über alle Läufe sehr stabil - das ist die reine Verarbeitungszeit eines einzelnen Requests ohne Mutex-Wartezeit. Die hohe Varianz bei p95 (30-52 s) und der Fehlerrate (26-35%) ist dagegen direkt auf Netzwerkschwankungen bei der Übertragung des 80 MB großen ServerKey zurückzuführen. Je nach Netzwerklast zum Messzeitpunkt verschiebt sich der Anteil der Requests, die den Proxy-Timeout überschreiten.

Die Throughput-Grenze liegt bei einem parallelen Request. Jeder weitere gleichzeitige Request verlängert die Wartezeit linear, da `tfhe::set_server_key` einen globalen Thread-Kontext setzt und die gesamte Verarbeitung (Übertragung + Dekomprimierung + FHE) serialisiert wird. Ab 2 VUs überschreitet p95 den Proxy-Timeout von 60 s regelmäßig.

Limitationen

- Der Use Case ist vollständig zustandslos. Es gibt keine Session, keine Audit-Log und keine Möglichkeit, Ergebnisse serverseitig zu speichern oder abzurufen.
- Der `CompressedServerKey` (80 MB) wird bei jedem einzelnen Request vom Client mitgeschickt, deserialisiert und dekomprimiert. Dieses Design ist eine direkte Konsequenz der Zustandslosigkeit: Da jeder Client sein eigenes Schlüsselpaar generiert, kann kein globaler ServerKey serverseitig gespeichert werden, ohne das Sicherheitsmodell zu brechen. Ein gespeicherter ServerKey eines anderen Clients würde es diesem ermöglichen, fremde Ergebnisse zu entschlüsseln. Die Folge ist, dass Netzwerkübertragung und Dekomprimierung die Latenz dominieren und ein produktiver Einsatz über das Internet praktisch nicht skaliert.
- Der Server prüft nicht, ob ein `encrypted_age`-Ciphertext bereits zuvor verwendet wurde. Ein Angreifer, der einen Ciphertext abfängt, kann ihn beliebig oft einreichen.
- Der Client übergibt den `CompressedServerKey` selbst. Ein bössartiger Client könnte einen manipulierten Key einreichen. In einer produktiven Umgebung müsste der ServerKey serverseitig fest hinterlegt sein.
- Der Client könnte einen beliebigen `FheInt8`-Wert übermitteln - der Server kann nicht verifizieren, dass die verschlüsselte Eingabe tatsächlich ein Alter darstellt oder aus einer vertrauenswürdigen Quelle stammt (kein Zero-Knowledge-Beweis).
- Maximal darstellbarer Alterswert ist 127 Jahre (`i8::MAX`). Dies ist für den Anwendungsfall ausreichend, aber die Wahl von `FheInt8` schließt größere Ganzzahlen strukturell aus.
- Es gibt keine Authentifizierung am Endpunkt. Jeder, der die API kennt, kann Anfragen stellen. Ein Gateway-Layer (z. B. API-Key, mTLS) ist in der aktuellen Implementierung nicht vorhanden.
- Durch den globalen `set_server_key`-Aufruf ist echte Parallelverarbeitung nicht möglich. Der Durchsatz skaliert nicht mit der Anzahl der CPU-Kerne.

4.3 Encrypted-Voting-&-Polling

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.4 Sealed-Bid-Auction

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.5 Encrypted-Statistics-Service

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.6 Encrypted-Genomics

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.7 Encrypted-Image-Processing

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.8 Encrypted-Leaderboard

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

4.9 Encrypted-Program-Execution

Funktionsbeschreibung

OpenAPI-Schnittstelle

Trust- und Threat-Model

FHE-Designentscheidungen

Komplexität der eigenen Algorithmen

Performance-Messung

Limitationen

A Anhang

Literatur

[Mus23] M. Mustermann. Example title. Online, 2023.